

Nama : Aura Rieska Maharani

NIM : 434241051

Kelas : C4

Github : https://github.com/aurarieska/434241051_tm1_backendlanjutbelajar

LAPORAN PRAKTIKUM MODUL 1

LANGKAH 1 — Instalasi dan Verifikasi

```
● PS C:\aura\sm5\backend\belajar\tm1> go version
go version go1.26.5 windows/amd64
● PS C:\aura\sm5\backend\belajar\tm1> git --version
git version 2.51.1.windows.1
● PS C:\aura\sm5\backend\belajar\tm1> psql --version
psql (PostgreSQL) 18.4
● PS C:\aura\sm5\backend\belajar\tm1> mongod --version
db version v8.3.8
Build Info: {
  "version": "8.3.8",
  "gitVersion": "35e8c8a57f78157ed9fac1a9e90ee6c1818adab6",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows",
    "distarch": "amd64",
    "target_arch": "amd64"
  }
}
```

LANGKAH 2 — Inisialisasi Project

```
● PS C:\aura\sm5\backend\belajar\tm1> go mod init
go: C:\aura\sm5\backend\belajar\tm1\go.mod already exists
```

LANGKAH 3 — Hello World dengan Fiber

```
package main // untuk pengelompokan package, file termasuk paket mana. dampaknya : bisa panggil function dalam package yg sama. klo beda gabisa.

import (
    "log"

    "github.com/gofiber/fiber/v2" // untuk import package fiber agar bisa pakai framework fiber untuk golang
)

func main() {
    app := fiber.New() // untuk membuat aplikasi/server fiber baru

    app.Get("/", func(c *fiber.Ctx) error {
        return c.SendString("Hello, World!") // untuk print hallo word, sebagai anda berhasil atau gak.
    })

    log.Fatal(app.Listen(":3000"))
}
```

Package = untuk pengelompokan package, file termasuk paket mana. dampaknya : bisa panggil function dalam package yg sama. klo beda gabisa. Catatan, harus di folder yg sama

Kode dari package berbeda juga dapat digunakan melalui import, selama kode tersebut exported.

Contoh :

```
project/
├── go.mod
├── kampus/
│   └── kampus.go
└── sekolah/
    └── sekolah.go
```

sekolah/sekolah.go :

```
Go
package sekolah

func Sekolah() {
    println("Ini function dari package sekolah")
}
```

Perhatikan `S` besar:

```
Go
Sekolah()
```

Karena huruf awalnya besar, function tersebut **exported**, sehingga package lain boleh menggunakannya.

Lalu `kampus/kampus.go` :

```
Go
package kampus

import "project/sekolah"

func Jalankan() {
    sekolah.Sekolah()
}
```

127.0.0.1:3000

127.0.0.1:3000

Hello, World!

LANGKAH 4 — Latihan Sintaks dalam Satu Berkas

```
package main

import "fmt" // import package fmt bawaan Go untuk formatting dan I/O

type User struct { //class
    ID      int    `json:"id"` // nama variabel, type data kolom, konversi ke json--> untuk bikin API
    Username string `json:"username"`
    IsActive bool   `json:"is_active"`
}

func (u *User) Activate() { u.IsActive = true }
func (u User) Info() string { return fmt.Sprintf("%s aktif: %v", u.Username, u.IsActive) }
```

`u *User` : pointer receiver menerima pointer/alamat ke data User asli. Perubahan pada dilakukan di alamat data asli sehingga memengaruhi data asli. Cocok jika method perlu mengubah data User.

`u user` : menerima salinan (copy) dari data. Perubahan pada `u` tidak memengaruhi data asli. Cocok jika method hanya perlu membaca/memproses data.

`fmt.print()` = print

`fmt.println` = print dengan newline

fmt.printf = print dengan format
fmt.sprint = menyimpan dalam string
fmt.sprintln = menyimpan dalam string dengan newline
fmt.sprintf = menyimpan string dengan format

penjelasan dengan format :

fmt.printf("%formatpilihan1 katakata %formatpilihan2 dst", value1, value2, dst)

%s = string

%d = integer

%f = float

%v = apa saja default

%t = boolean

%c = karakter

%x = hexadecimal --> 255->ff

%e = notasi ilmiah --> 3.14e+00

Tipe data asli	Format yang cocok
int	%d %b %o %O %x %X %c %U %q %v
float32 , float64	%f %e %E %g %G %x %X %v
string	%s %q %x %X %v
bool	%t %v
rune	%d %b %o %O %x %X %c %U %q %v
byte	%d %b %o %O %x %X %c %U %q %v
semua tipe	%v %T

Format	Keterangan	Contoh
%d	Integer dalam desimal (basis 10)	65 → 65
%b	Integer dalam biner (basis 2)	65 → 1000001
%o	Integer dalam oktal (basis 8)	65 → 101
%O	Integer dalam oktal dengan prefix 0o	65 → 0o101
%x	Integer/byte/string dalam hexadecimal huruf kecil	65 → 41
%X	Integer/byte/string dalam hexadecimal huruf besar	65 → 41
%c	Integer/rune sebagai karakter Unicode	65 → A
%U	Integer/rune sebagai Unicode code point	65 → U+0041
%q	Nilai dalam bentuk quoted/berpetik	"Aura" → "Aura"
%s	String sebagai teks biasa	"Aura" → Aura
%f	Floating-point dalam notasi desimal	3.14 → 3.140000
%e	Floating-point dalam notasi ilmiah, e kecil	3.14 → 3.140000e+00
%E	Floating-point dalam notasi ilmiah, E besar	3.14 → 3.140000E+00
%g	Floating-point dalam format paling ringkas	3.14 → 3.14
%G	Sama seperti %g, tetapi menggunakan E jika notasi ilmiah	3.14 → 3.14
%t	Boolean	true → true
%v	Format default sesuai tipe datanya	65 → 65
%T	Menampilkan tipe data	65 → int

1. VARIABEL

```
func tukarNilai(a, b *int) { *a, *b = *b, *a }
func main() {
    var nama string = "Sari"
    umur := 20
    var kosong string
    fmt.Printf("%s %d zero value: %q\n", nama, umur, kosong)
```

cara deklarasi variabel :

1. var dengan tipe data dan value
var namaVariabel tipeData = value

2. Var tanpa value
var namaVariabel tipe data

Variabel akan otomatis dapat zero value sesuai tipe datanya

contoh :

int -> 0

float -> 0

string -> ""

bool -> false

3. var tanpa menuliskan tipe data
var namaVariabel = value

4. var dalam fungsi
namaVar = value

Go akan otomatis menentukan tipe data berdasarkan value yang diberikan

Namun, overwrite hanya bisa dilakukan dengan value bertipe data sama.

Jika overwrite beda tipe data maka akan error.

2. SLICE

Array dengan panjang fleksibel. Tipe data harus sama dan harus disebutkan saat deklarasi

```
nilai := []int{10, 20, 30}
nilai = append(nilai, 40)
fmt.Println(nilai, nilai[1:3], len(nilai), cap(nilai))
```

contoh :

array = [3]int

slice = []int

cara bikin slice :

1. literal

namaVar := []int(value1, value2, ...)

2. `make()` --> fokus utama untuk bikin slice kosong yg baru diketahui panjang sementara. Diisi zero value
`namaVar := make([]int, panjang)`

3. slice kosong
`namaVar := []int{}` di awal 1 ruang diisi zero value

related function:

1. `append()` --> menambah di indeks paling belakang
`namaVar = append(var, value)`
bisa juga klo mau tambah di tengah,
`namaVar = append(slice[:index], append([]Typedata{value}, slice[index:]...)...)`
konsepnya masukin ke indeks sebelum add, indeks add, dan indeks setelah add

untuk hapus, gaada fungsi khusus
solusinya adalah overwrite dengan slice baru tanpa elemen yang mau dihapus
`namaVar = append(namaVar[:indekshapus], namaVar[indekshapus+1:]...)`
konsepnya masukin indeks sebelum hapus dan indeks setelah hapus. jadi indeks hapus gak dimasukin

2. `len()` -> hitung panjang slice
`len(namaVar)`

3. `cap()` -> hitung kapasitas slice
`cap(namaVar)`

4. mengakses
`namaVar[index]`

5. mengambil sebagian (slicing)
`namaVar[indeksAwal:indeksAkhir]`

`len()` vs `cap()`

contoh :

`s := make([]int, 3, 5)`

`len = 3` → sekarang ada 3 elemen

`cap = 5` → slice punya kapasitas sampai 5 elemen

`[ 0 ][ 0 ][ 0 ][ ] [ ]`

←———— len 3 —————→

←———— cap 5 —————→

3. MAP

array dengan key-value.

syarat = key unik, value bisa sama

```
skor := make(map[string]int)
skor["Sari"] = 90
if n, ada := skor["Budi"]; ada {
    fmt.Println("Budi:", n)
} else {
    fmt.Println("Budi belum punya nilai")
}
```

deklarasi :

namaVar := map[keyType]valueType

process related :

1. menambah

namaVar[Key] = Value

2. akses

namaVar[Key]

3. Delete

delete(namaVar, Key)

4. cek ada atau tidak

namaVarValue, namaVarBoolean := namaMap[key]

4. CHANNEL

media komunikasi antar goroutine

goroutine = fungsi yang dijalankan secara concurrent oleh Go menggunakan keyword.

Intinya biar gak jalan bareng. Misal fungsi A dan B run barengan. Tidak nunggu A selesai baru B

```
ch := make(chan string)
go func() { ch <- "halo dari goroutine" }()
fmt.Println(<-ch)
```

5. POINTER

untuk menunjuk alamat asli data --> dikenai proses data asli bukan salinan

```
a, b := 1, 2
tukarNilai(&a, &b)
fmt.Println("setelah ditukar:", a, b)
```

& = alamat atau indeks

* = nilai yg berada di alamat

Contoh :

&value = alamat asli dari value

*a = nilai asli variabel a

6. STRUCT

Bagian deklarasi struct atau yg mirip kelas dalam php ini sudah dijelaskan di atas, bagian ini menjelaskan cara memberi struct value. Setiap struct bervalue harus disimpan ke dalam variabel berbeda.

Format :

namaVariabel := namaStruct{kolom1: value1, kolom2: value2, dst}

```
u := User{ID: 1, Username: "sari"}
u.Activate()
fmt.Println(u.Info())
```

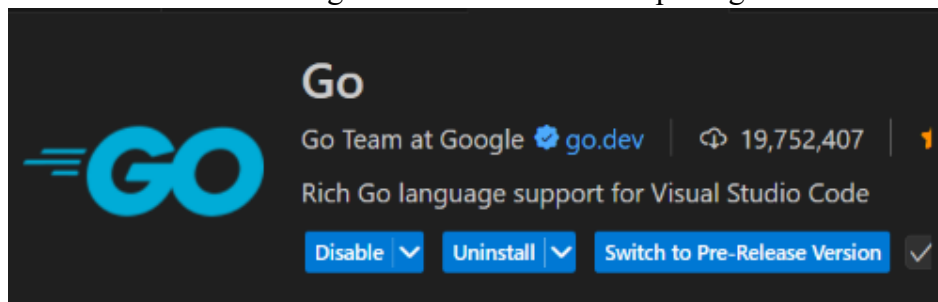
TUGAS MANDIRI

1. Bukti Persiapan Lingkungan

- Hasil go version, git --version, psql --version, dan mongod --version

```
PS C:\aura\sm5\backend\belajar\tm1> go version
go version go1.26.5 windows/amd64
PS C:\aura\sm5\backend\belajar\tm1> git --version
git version 2.51.1.windows.1
PS C:\aura\sm5\backend\belajar\tm1> psql --version
psql (PostgreSQL) 18.4
PS C:\aura\sm5\backend\belajar\tm1> mongod --version
db version v8.3.8
Build Info: {
  "version": "8.3.8",
  "gitVersion": "35e8c8a57f78157ed9fac1a9e90ee6c1818adab6",
  "modules": [],
  "allocator": "tcmalloc-gperf",
  "environment": {
    "distmod": "windows",
    "distarch": "amd64",
    "target_arch": "amd64"
  }
}
```

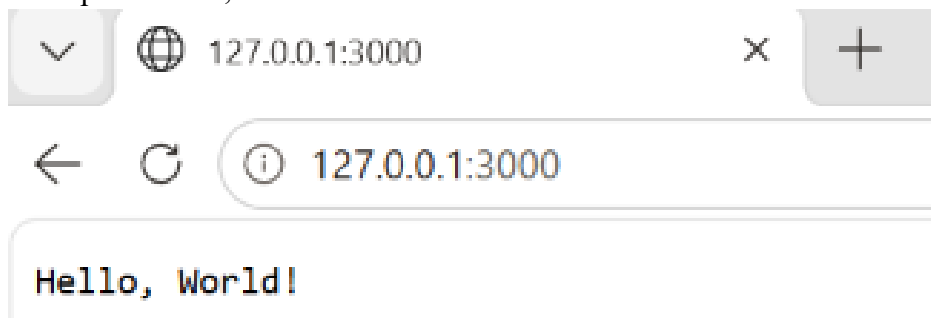
- Visual Studio Code dengan ekstensi Go sudah terpasang



- Terminal setelah menjalankan go mod init

```
PS C:\aura\sm5\backend\belajar\tm1> go mod init
go: C:\aura\sm5\backend\belajar\tm1\go.mod already exists
```

- Tampilan Hello, World! di browser



2. Variabel dan Struktur Data

- Deklarasi lima variabel dengan tipe berbeda: string, int, float64, bool, dan slice

```
package main // 2. Deklarasi lima variabel dengan tipe berbeda: string, int, float64, bool, dan slice

// STRING
var akuStringTp string = "Aura Rieska Maharani"
var akuString = "Aura Rieska Maharani"
// hasil aja declrasi maupun tidak deklarasi tipe data, bedanya klo type data dideklarasikan sejak awal maka akan strict dan menolak value yg bukan string
// eks : var akuString String = 20 --> ditolak karena integer, tidak sesuai tipe data yang dideklarasikan.
var stringKosong string // zero value string = "", harus sebutin tipe data agar tau zero valuenya

// INT
var iniIntTp int = 20
var iniInt = 21
var intKosong int

// FLOAT
var ituFloatTp float64 = 23.923892839
var yoFloat = 28.30290
var floatKosong float64

// BOOLEAN
var buyianTp bool = true
var bulian = false
var boolKosong bool

// SLICE
// slice literal
var slcliteral = []int{1, 33, 2323, 433, 32, 445}

// make zero value --> menyebutkan panjang awal, semua diisi zero value
var slicemake = make([]int, 7)

// slice kosong --> di awal 1 ruang diisi zero value
var sliceKosong = []int{}
```

```
package main

import "fmt"

func main() {
    println("string dengan tipe data dan value ", akuStringTp)
    println("string tanpa tipe data dengan value ", akuString)
    println("string tanpa tipe data dan value ", stringKosong)
    println("int dengan tipe data dan value ", iniIntTp)
    println("int tanpa tipe data dengan value ", iniInt)
    println("int tanpa tipe data dan value ", intKosong)
    println("float dengan tipe data dan value ", ituFloatTp)
    println("float tanpa tipe data dengan value ", yoFloat)
    println("float tanpa tipe data dan value ", floatKosong)
    println("boolean dengan tipe data dan value ", buyianTp)
    println("boolean tanpa tipe data dengan value ", bulian)
    println("boolean tanpa tipe data dan value ", boolKosong)
    fmt.Println("slice dengan tipe data dan value ", slcliteral)
    fmt.Println("slice tanpa tipe data dengan value ", slicemake)
    fmt.Println("slice tanpa tipe data dan value ", sliceKosong)
}
```

Variabel-variabel yang telah dideklarasikan tadi dipanggil pada function main untuk di print dengan command println, kecuali pada variabel slice harus menggunakan fmt.println untuk menyesuaikan format print slice.

Hasil :

```
PS C:\aura\sm5\backend\belajar\tm1\tugasmandiri> go run .
string dengan tipe data dan value  Aura Rieska Maharani
string tanpa tipe data dengan value  Aura Rieska Maharani
string tanpa tipe data dan value
int dengan tipe data dan value  20
int tanpa tipe data dengan value  21
int tanpa tipe data dan value  0
float dengan tipe data dan value  23.923892839
float tanpa tipe data dengan value  28.3029
float tanpa tipe data dan value  0
boolean dengan tipe data dan value  true
boolean tanpa tipe data dengan value  false
boolean tanpa tipe data dan value  false
slice dengan tipe data dan value  [1 33 2323 433 32 445]
slice tanpa tipe data dengan value  [0 0 0 0 0 0]
slice tanpa tipe data dan value  []
```

3. Pointer

perbandingan pass by value dengan pass by pointer

```
func swapbypointer(a, b *int){ /*int artinya alamat dari variabel bertipe int
// jadi parameter function ini harus berupa alamatnya
    var temp1 = *a
    *a=*b
    *b=temp1
}

func swapbyvalue(a, b int){
    var temp2 = a
    a=b
    b=temp2
}
```

swapbypointer melakukan pertukaran nilai pada variabel di main melalui alamat yang dikirim sebagai parameter.

Proses:

main mengirim alamat variabel ke swapbypointer → swapbypointer menerima alamat tersebut melalui parameter pointer → swapbypointer mengakses nilai asli menggunakan * → nilai kedua variabel di main ditukar.

swapbyvalue hanya melakukan pertukaran pada parameter a dan b yang merupakan salinan nilai dari variabel di main, sehingga perubahan tersebut tidak memengaruhi variabel asli di main.

Proses :

main mengirim nilai variabel ke swapbyvalue → swapbyvalue menerima nilai tersebut sebagai salinan pada parameter a dan b → swapbyvalue menukar nilai a dan b yang hanya merupakan salinan → setelah function selesai, salinan tersebut tidak dibawa kembali ke main, sehingga nilai variabel asli di main tetap.

```

a := 10
b := 100

println("sebelum swapbypointer", a, b)
swapbypointer(&a,&b)
println(a, b)

//buat variabel baru karena yg atas sudah kena overwrite
c := 10
d := 100
println("sebelum swapbyvalue", c, d)
swapbyvalue(c,d)
println(c,d)

```

Saat memanggil fungsi swapbypointer, parameter adalah alamat di main. Sedangkan saat memanggil fungsi swapbyvalue adalah nama variabel

Untuk menambah item bar uke slice, gunakan function append() bawaan go yang dideklarasikan dengan parameter:

1. Variabel menyimpan alamat bertipe sesuai tipe data slice
2. NamaVarValue

```

func updateslice(s *[]string, newItem string){
    *s = append(*s, newItem)
}

```

Saat fungsi dipanggil, parameternya diisi dengan &namaVarSlice yang berisi alamat slice dan value yang ingin dimasukkan

```

e := []string{"aura", "rieska"}
fmt.Println("sebelum updateslide", e)
updateslice(&e, "maharani")
fmt.Println("sesudah updateslide", e)
}

```

4. Struct Student

```

type Student struct {
    ID int           `json: "id"`
    Name string       `json: "nama"`
    Grade float64     `json: "kelas"`
    IsActive bool     `json: "status"`
}

```

Deklarasi struct beserta namaKolom setiap kolom dan tipe datanya.

```
// deklarasi object Student
var siswa1 = Student{
    ID : 1,
    Name : "Aura",
    Grade : 100,
    IsActive : false,
}

// deklarasi object (struct dengan value) lebih dari 1
var siswa2 = Student{
    ID:      2,
    Name:     "Sari",
    Grade:    90,
    IsActive: true,
}

var siswa3 = Student{
    ID:      3,
    Name:     "Yaya",
    Grade:    85,
    IsActive: false,
}
```

Kemudian isi value struct dan simpan pada Variabel baru (object).

```
func GetInfo() string {
    info := fmt.Sprintf(siswa1.ID, siswa1.Name, siswa1.Grade, siswa1.IsActive)
    return info
}
```

GetInfo() mendapatkan object (struct yg memiliki value) dengan fmt.Sprintf dan return karena requirement soal minta tanpa parameter sehingga nama VarObject harus disebutkan dan hanya bisa menampilkan 1 object.

```
func main() {

    println(GetInfo())
}
```

Penggunaannya cukup dengan call function biasa, agar bisa ditampilkan dimasukkan sebagai parameter println

Hasil :

```
1 Aura 100 false
```

```

func UpdateGrade(grade float64){
    siswa1.Grade = grade
}

func activate(){
    siswa1.IsActive = true
}

func deactivate(){
    siswa1.IsActive = false
}

```

Untuk update nilai,

Karena pada requirement soal diminta parameter grade maka untuk memperbarui nilai dilakukan dengan overwrite kolom grade dengan menyebutkan juga nama VarObject dengan nama Var parameter.

Untuk activate dan deactivate,

Karena parameter kosong, jadi disebutkan nama VarObject dan kolom isactivate dengan value true dan false.

Beda antara UpdateGrade dengan activate dan deactivate adalah overwrite dilakukan dengan nilai dari suatu Variabel, sedangkan activate dan deactivate di overwrite dengan nilai yang dideklarasikan langsung dalam function

```

// Update melalui input di luar function karena requirement soal
println("masukkan nilai baru")
var nilai float64
fmt.Scan(&nilai)
UpdateGrade(nilai)
println("status setelah melalui update nilai")
println(GetInfo())

println("status setelah melalui activate")
activate()
println(GetInfo()) // You, 15 hours ago • no 4, struct student

println("status setelah melalui deactivate")
deactivate()
println(GetInfo())

```

Untuk update Scan nilai dilakukan lewat main karena parameter meminta value nilai baru kemudian panggil function updateGrade dengan parameter berupa nama Variabel yang menyimpan value nilai baru. Sedangkan function activate dan deactivate cukup panggil function karena value overwritenya fixed declared melalui function.

Hasil :

```

masukkan nilai baru
99
status setelah melalui update nilai
1 Aura 99 false

status setelah melalui activate
1 Aura 99 true

status setelah melalui deactivate
1 Aura 99 false

```

SKENARIO TAMBAHAN JIKA OBJECT LEBIH DARI 1

```
// buat fungsi getinfo baru dengan parameter namaObject
func GetInfoStudent(siswa Student) string {
    info := fmt.Sprintf(
        siswa.ID,
        siswa.Name,
        siswa.Grade,
        siswa.IsActive,
    )
    return info
}
```

Untuk memudahkan menampilkan seluruh object dari struct, bisa ditambahkan parameter variabel yang nantinya akan dioverwrite melalui main dengan namaVarObject yang ingin diambil infonya.

```
// getinfo dengan parameter karena struct banyak jadi harus disebutin namaObjectnya
fmt.Println("Informasi Siswa 1:")
fmt.Println(GetInfoStudent(siswa1))

fmt.Println("Informasi Siswa 2:")
fmt.Println(GetInfoStudent(siswa2))

fmt.Println("Informasi Siswa 3:")
fmt.Println(GetInfoStudent(siswa3))
```

=== TANPA LOOPING ===

Informasi Siswa 1:

1 Aura 99 false

Informasi Siswa 2:

2 Sari 90 true

Informasi Siswa 3:

3 Yaya 85 false

Jika ingin panggil semua object bisa otomatis melalui looping, namun untuk melakukan looping harus deklarasi variabel slice untuk iterasi objectnya

```
// deklarasi object dalam slice untuk looping
var daftarSiswa = []Student{
    siswa1,
    siswa2,
    siswa3,
}
```

Tidak perlu parameter karena yg diprint adalah semua bukan object tertentu sehingga tidak perlu parameter untuk memilih/menentukan object mana yg harus di print

```
// Menampilkan semua student dengan looping
func GetAllStudent() {
    for i, siswa := range daftarSiswa {
        fmt.Println("Student ke-", i+1)
        fmt.Println("ID:", siswa.ID)
        fmt.Println("Nama:", siswa.Name)
        fmt.Println("Grade:", siswa.Grade)
        fmt.Println("Status:", siswa.IsActive)
        fmt.Println()
    }
}
```

```
/*
Object Struct Banyak dengan looping
*/

fmt.Println("=== DENGAN LOOPING ===")

GetAllStudent()
```

Hasil :

```
=== DENGAN LOOPING ===
Student ke- 1
ID: 1
Nama: Aura
Grade: 100
Status: false

Student ke- 2
ID: 2
Nama: Sari
Grade: 90
Status: true

Student ke- 3
ID: 3
Nama: Yaya
Grade: 85
Status: false
```

Untuk update pada object tertentu harus ditambahkan parameter namaVar yg nantinya akan di overwrite di main dengan namaVarObject yang ingin dilakukan updating

```
// Mengubah grade object Student tertentu
func UpdateGradeStudent(siswa *Student, grade float64) {
    siswa.Grade = grade
}

// Mengaktifkan student tertentu
func ActivateStudent(siswa *Student) {
    siswa.IsActive = true
}

// Menonaktifkan student tertentu
func DeactiveStudent(siswa *Student) {
    siswa.IsActive = false
}
```

Untuk penerapannya,

```
//looping grade setiap object diperbarui dengan nilai berbeda
fmt.Println("=== UPDATE NILAI SEMUA SISWA ===")

for i := range daftarSiswa {
    var nilaiBaru float64

    //scan dimasukkan di dalam looping dan perbarui nilai
    fmt.Print("Masukkan nilai untuk ", daftarSiswa[i].Name, ": ")
    fmt.Scan(&nilaiBaru)

    UpdateGradeStudent(&daftarSiswa[i], nilaiBaru)
}

GetAllStudent()
```

jika setiap object dari struct siswa updatenya nilainya berbeda, maka scan nilai dan pemanggilan function updatenya di dalam looping

```
=== UPDATE NILAI SEMUA SISWA ===
Masukkan nilai untuk Aura: 99
Masukkan nilai untuk Sari: 98
Masukkan nilai untuk Yaya: 78
Student ke- 1
ID: 1
Nama: Aura
Grade: 99
Status: false

Student ke- 2
ID: 2
Nama: Sari
Grade: 98
Status: true

Student ke- 3
ID: 3
Nama: Yaya
Grade: 78
Status: false
```

```

//looping grade setiap object diperbarui dengan nilai sama
fmt.Println("=== UPDATE NILAI DISAMAKAN SEMUA SISWA ===")
var nilaiBaruSama float64

//nilai dimasukkan sebelum looping dan disimpan dalam variabel nilaiBaruSama
fmt.Print("Masukkan nilai baru ")
fmt.Scan(&nilaiBaruSama)

//looping hanya untuk perbarui nilai setiap object dengan nilai yang sama dari nilaiBaruSama
for i := range daftarSiswa {
    UpdateGradeStudent(&daftarSiswa[i], nilaiBaruSama)
}

GetAllStudent()
}

```

Jika update nilainya disamakan semua object siswa nilainya sama berdasarkan value input maka scan dilakukan sebelum looping, looping hanya memanggil function untuk memperbarui kolom grade setiap object dari struct siswa.

```

=== UPDATE NILAI DISAMAKAN SEMUA SISWA ===
Masukkan nilai baru 100
Student ke- 1
ID: 1
Nama: Aura
Grade: 100
Status: false

Student ke- 2
ID: 2
Nama: Sari
Grade: 100
Status: true

Student ke- 3
ID: 3
Nama: Yaya
Grade: 100
Status: false

```

MAIN POINT

& variabel = mengubah isi alamat (isi alamat variabel asli terdampak)

Variabel = hanya ambil saja (Salinan cukup)

fmt.Scan(&namaVar) karena scan butuh alamat buat nyimpan